



ugr

Universidad
de Granada

TRABAJO DE ASIGNATURA PDIH

PARKING INTELIGENTE

Autor:

Mario Casas Pérez

ÍNDICE

Introducción	3
Motivación	6
Diseño del parking	8
Parking montado con Tinkercad	12
Esquema Fritzing	20
Parking montado físicamente con Arduino IDE	21
Maqueta y funcionamiento	24
Conclusiones	26
Bibliografía	27

Introducción

En la actualidad, la congestión urbana y la escasez de espacios de estacionamiento son problemas clave que afectan a varias ciudades, generando ineficiencias y frustraciones para los conductores. Para solucionar este problema, mi proyecto se basa en la construcción de un parking inteligente en Arduino y en el simulador Tinkercad.

El uso de Tinkercad me ha permitido diseñar, simular y validar el circuito a través de una interfaz cómoda e intuitiva, tanto para montar el proyecto como en la búsqueda de los componentes para su montaje. Gracias a esta herramienta no es necesario tener un hardware físico en las primeras etapas de la construcción, por lo que me ha ayudado para ir modelando poco a poco el diseño y construcción del parking para, posteriormente, poder adaptarlo fácilmente a las necesidades físicas de su montaje.

Por otro lado, una vez hecho el parking en Tinkercad, se ha pasado la programación en código a Arduino IDE para poder cargar el programa con los componentes del parking montados físicamente. Arduino me ha ayudado a cargar la parte lógica y operativa del proyecto, dando la capacidad de procesamiento necesario para gestionar en tiempo real la información de los sensores y ejecutar todas las acciones programadas correctamente, así como la actualización del estado de cada plaza disponible o la activación de las alertas impuestas bajo ciertas condiciones.

Mi idea para este proyecto ha sido montar un parking el cual está limitado a 5 plazas de coche. Se dispondrá de una pantalla lcd 16x2 I2C en donde se indicará en la puerta de entrada del parking el número total de plazas, la temperatura ambiente del parking, y diversos mensajes que saldrán conforme se cumplan distintos criterios. Para su funcionamiento, lo que se hace es que si un coche se encuentra cerca de la barrera de entrada del parking y pulsa el botón de entrada, se abrirá esta barrera indicando un mensaje de bienvenida y actualizando el número de plazas disponibles actuales. En el caso de que saliera algún coche, este deberá de estar cerca de la barrera de salida y pulsar el botón de salida para que dicha barrera se abra.

Los componentes usados han sido los siguientes:

- **Arduino UNO R3:** usado para conectar los componentes con sus respectivos pines.
- **2 placas de pruebas:** se ha usado para conectar los componentes con el Arduino UNO R3.
- **2 pulsadores:** uno para entrada y otro para salida. Si se presiona estando el coche cerca de la barrera, entonces esta se abrirá.
- **2 servomotores:** simularán las barreras de entrada y de salida. Si se cumple la condición de que el coche está cerca del servo (ya sea entrada o salida) y se presiona el pulsador, entonces la barrera se abrirá.

- **2 sensores de movimiento ultrasónicos:** uno para entrada y otro para salida. Debemos de comprobar que el coche esté lo suficientemente cerca de la barrera (servomotor) para que pulse el pulsador y esta se abra (lo mismo para la puerta de salida).
- **1 LED rojo:** se usará para la entrada, indicando al coche que la barrera está cerrada y que no puede pasar al parking.
- **1 LED verde:** se usará para la entrada, indicando al coche que puede entrar al parking (barrera levantada).
- **2 resistencias de 220Ω:** usadas en cada ánodo de los LEDs para evitar sobretensiones en los diodos LED.
- **2 resistencias de 1kΩ:** usadas en los pulsadores para evitar lecturas erróneas o fluctuaciones de voltaje en las entradas digitales cuando el botón no se encuentra presionado.
- **1 sensor de temperatura:** indicará en todo momento la temperatura ambiente presente en el parking.
- **1 piezo:** en el caso de que la temperatura del parking sea superior a un umbral de temperatura extrema, entonces empezará a pitar, indicando riesgo de incendio.
- **1 LCD 16x2 I2C:** se encuentra en la entrada del parking e indica el número de plazas disponibles y la temperatura ambiente. Además, se muestran mensajes adicionales si el coche se encuentra cerca de la barrera, indicando que pulse el botón y dándole la bienvenida. Además, en el caso de que el sensor de temperatura supere el umbral de temperatura extrema, saldrá un mensaje indicando que puede haber incendio en el parking. Además, si el parking se encuentra completo, también se indicará, impidiendo la entrada a más automóviles.
- **Cables:** para conectar los componentes electrónicos entre sí y realizar las conexiones con la placa Arduino UNO R3.

Con todos estos componentes se podrá realizar el proyecto de la asignatura correctamente, en donde tendremos un parking con 2 barreras (simuladas con servomotores), una de entrada y otra de salida.

- **Entrada:** si un coche se encuentra cercano a la barrera, se mostrará un mensaje adicional en el LCD indicando que pulse el botón de entrada. Una vez se presione el pulsador, la barrera se levantará y el LED verde se encenderá, mostrándose en el LCD un mensaje de bienvenida y actualizando el número de plazas disponibles en el parking. Una vez el coche entre, se cierra la barrera.
- **Salida:** en el caso de que un coche quiera salir del parking, deberá de presionar el pulsador de salida. Una vez hecho, la barrera de salida se abrirá y el coche podrá salir, actualizándose en el LCD el número de plazas disponibles.

- **Temperatura:** todo el tiempo se estará midiendo la temperatura del parking, mostrándola en el LCD. En el caso de que se supere el umbral de temperatura extrema, entonces se indicará en el LCD que hay riesgo de incendio, abriéndose la barrera de salida para que los coches puedan salir (mientras vayan saliendo se siguen actualizando el número de plazas disponibles aunque no se presione el pulsador de salida).

En las siguientes secciones se explicará con detalle el proyecto realizado, así como el uso de los componentes, la construcción del parking, así como del funcionamiento de este y el código realizado para poder llevarlo a cabo.

Motivación

Mi motivación para llevar a cabo este proyecto en la asignatura de PDIH se debe a mi interés de saber el funcionamiento de un parking, en donde se deben de tener en cuenta muchas variables y en donde todo debe de hacerse a tiempo real para que haya problemas a la hora de las plazas totales de este.

Además, pienso que este proyecto da respuesta a un problema real (aunque hecho a maqueta), en donde es importante tener en cuenta dicha gestión automática y optimizada en los parkings actuales, los cuales se deben de tener un sistema integrado en tiempo real y en donde cada variable y componente debe interactuar de forma precisa y coordinada para que no surja ningún fallo.

Parte de los conocimientos adquiridos en el seminario 3 y práctica 3 de esta asignatura me ha ayudado a aplicar de manera práctica los conocimientos adquiridos en dichas sesiones, combinando tanto la teoría como la práctica para la realización de este proyecto. El uso de herramientas como Tinkercad y Arduino nos permite diseñar, simular y validar circuitos y códigos en un entorno digital antes de hacerlo físicamente. Este proceso de validación virtual es muy importante ya que podemos ver como debería de funcionar los componentes físicos a través de un software preciso, además de que podemos experimentar con varias configuraciones y detectar posibles errores o mejoras sin necesidad de recurrir a costes elevados ni enfrentar riesgos con hardware real (ayudándonos y dando un enfoque metódico y cuidadoso en la implementación del proyecto).

Por otro lado, el paso de la simulación a la implementación física mediante el programa de Arduino IDE es la mejor forma de ver el reto que supone llevar a la práctica la lógica del funcionamiento del parking. Gestionar en tiempo real la información proveniente de sensores de movimiento, temperatura y otros dispositivos, reaccionando ante diferentes estímulos (presencia de un vehículo o superación de un umbral de temperatura) me ha ayudado a adquirir conocimientos para resolver problemas de este tipo, adquiriendo habilidades importantes en el área de Arduino. Esto me ha motivado a profundizar más en conceptos de electrónica y de programación.

Otra parte la cual es importante destacar es cómo la implementación de un sistema que monitoriza en tiempo real el estado del estacionamiento, actualice la información automáticamente en la pantalla LCD, y actúe de acuerdo a las señales registradas, me ha hecho comprender como pequeñas intervenciones son cruciales en la eficiencia de los recursos de los parkings, haciéndome entender que la mejora de la usabilidad y seguridad puede escalar a mayores niveles e incluso poder adaptarlos a entornos reales.

También me llamaba mucho la atención el entender el funcionamiento y la interacción de los componentes tecnológicos. A la hora de tener que ver el reto que supone el configurar sensores, actuadores y la pantalla LCD, he podido aprender más sobre el diseño de circuitos y la programación de microcontroladores. El desarrollo de este proyecto me ha motivado a seguir investigando en el campo de la electrónica y la automatización de

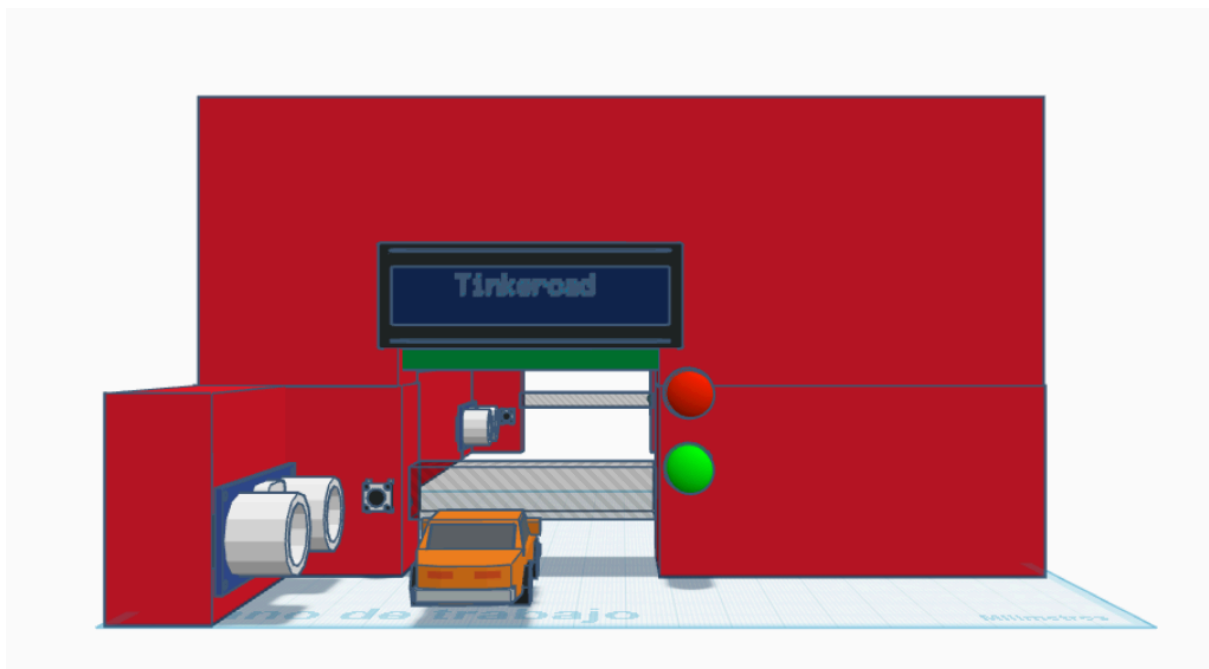
proyectos, los cuales son muy relevantes en la calidad de vida de las personas y en varios aspectos del funcionamiento de las ciudades.

Diseño del parking

Para el diseño del parking he hecho uso del simulador de Tinkercad, donde hay un apartado para crear diseños en 3D. Esta interfaz nos proporciona el poder crear un proyecto de manera virtual para poder ver la representación que este pudiera tener una vez montado físicamente.

La realización del parking, de las plazas del mismo y las barreras de entrada y salida, se han hecho a partir de formas básicas, simulando los muros con prismas rectangulares. Los componentes de Arduino (a excepción del piezo, dado a que como no se ha encontrado, lo he simulado con una semiesfera) se han podido añadir gracias a la parte de *Electrónica*, donde se encuentra gran parte de componentes electrónicos que se pueden usar en los circuitos en el simulador Tinkercad. Por otro lado, se han añadido figuras de coches para hacer más realista el parking.

- **FRONTAL**



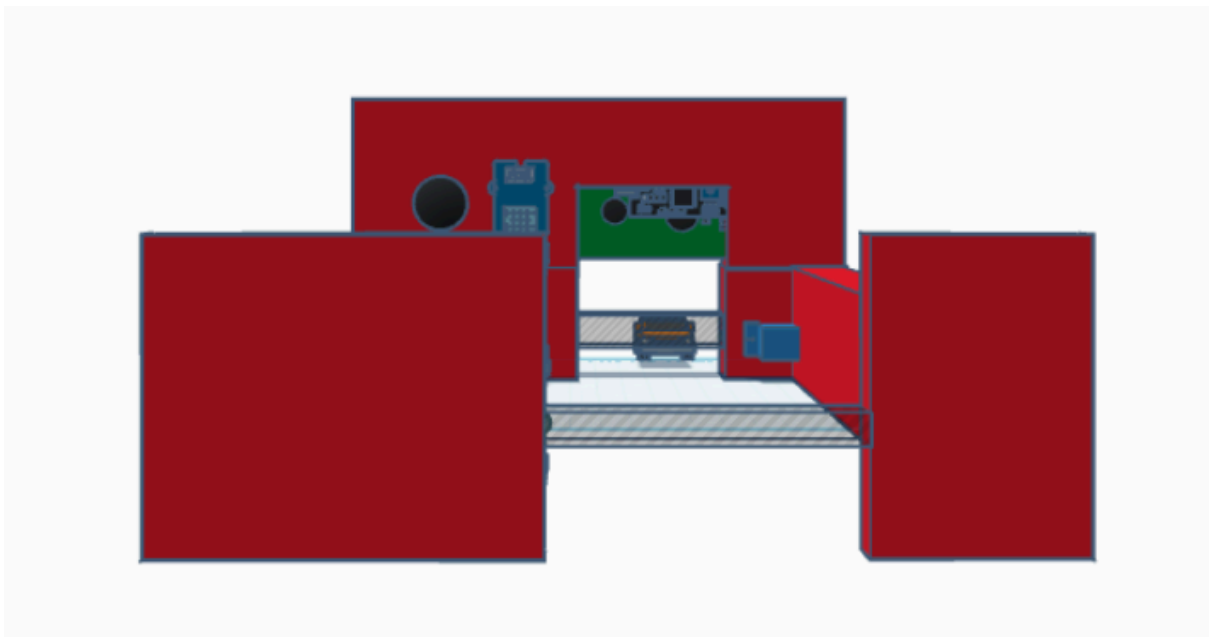
- IZQUIERDA



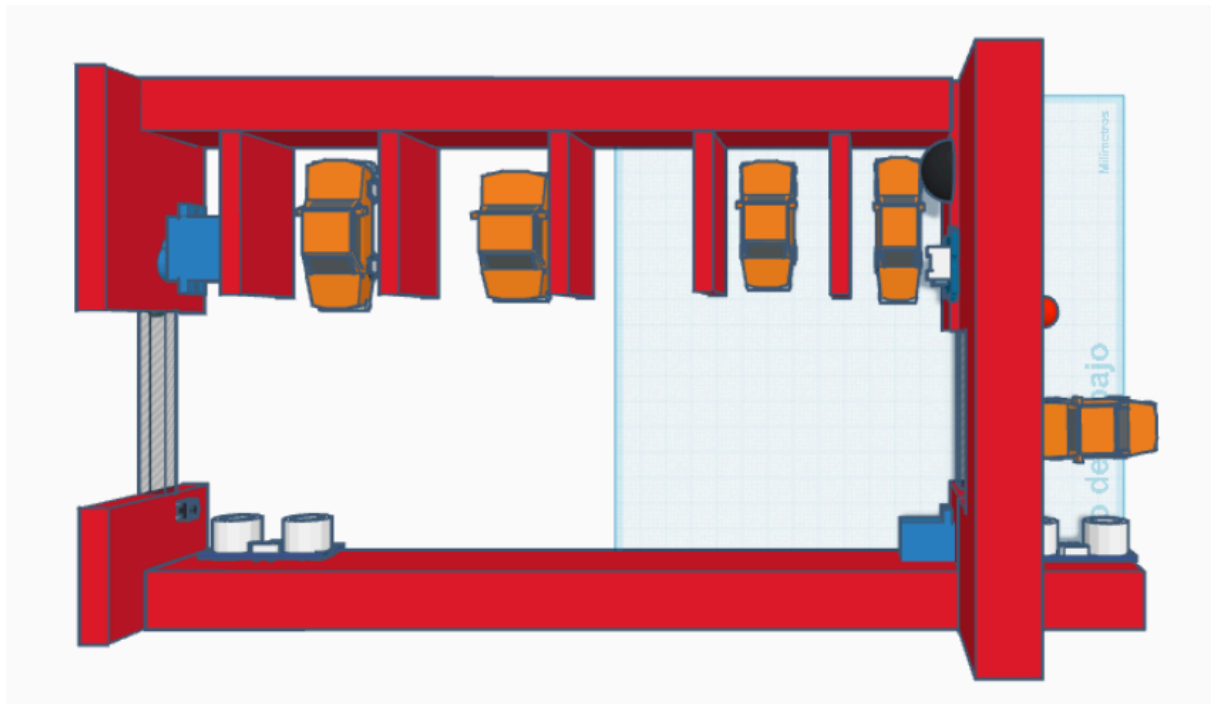
- DERECHA



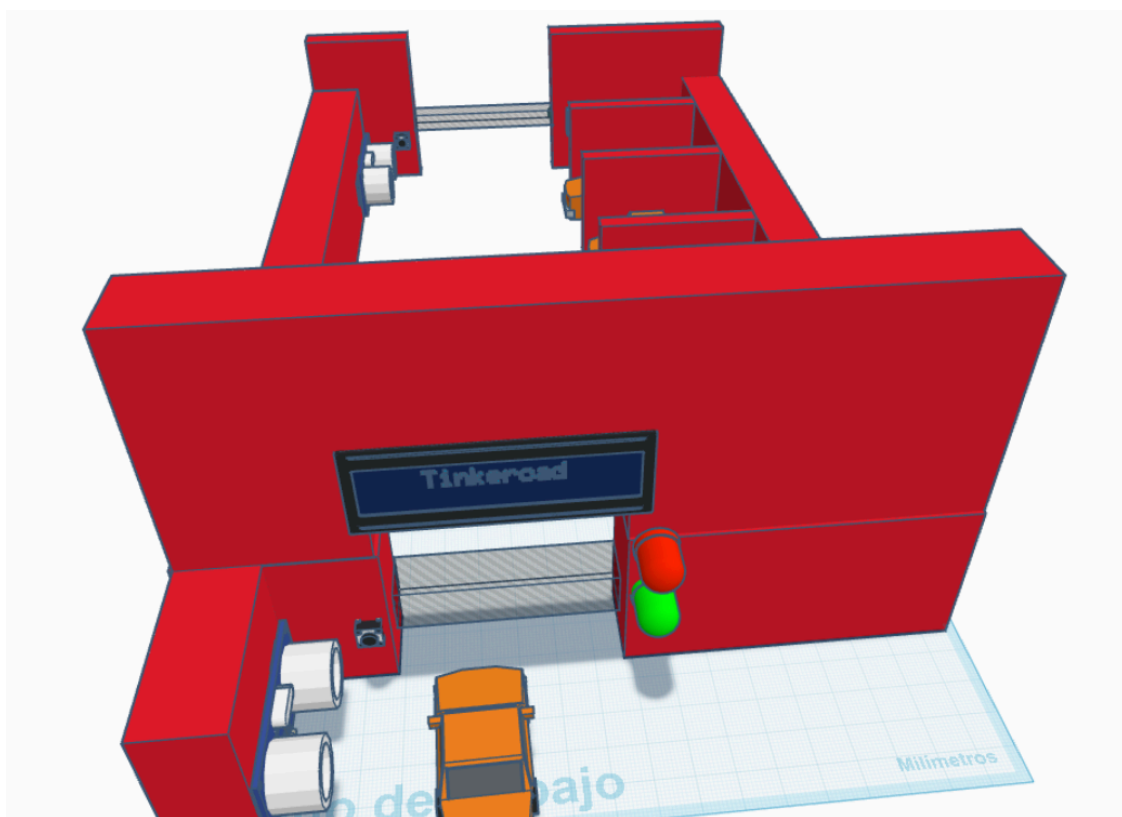
- DETRÁS



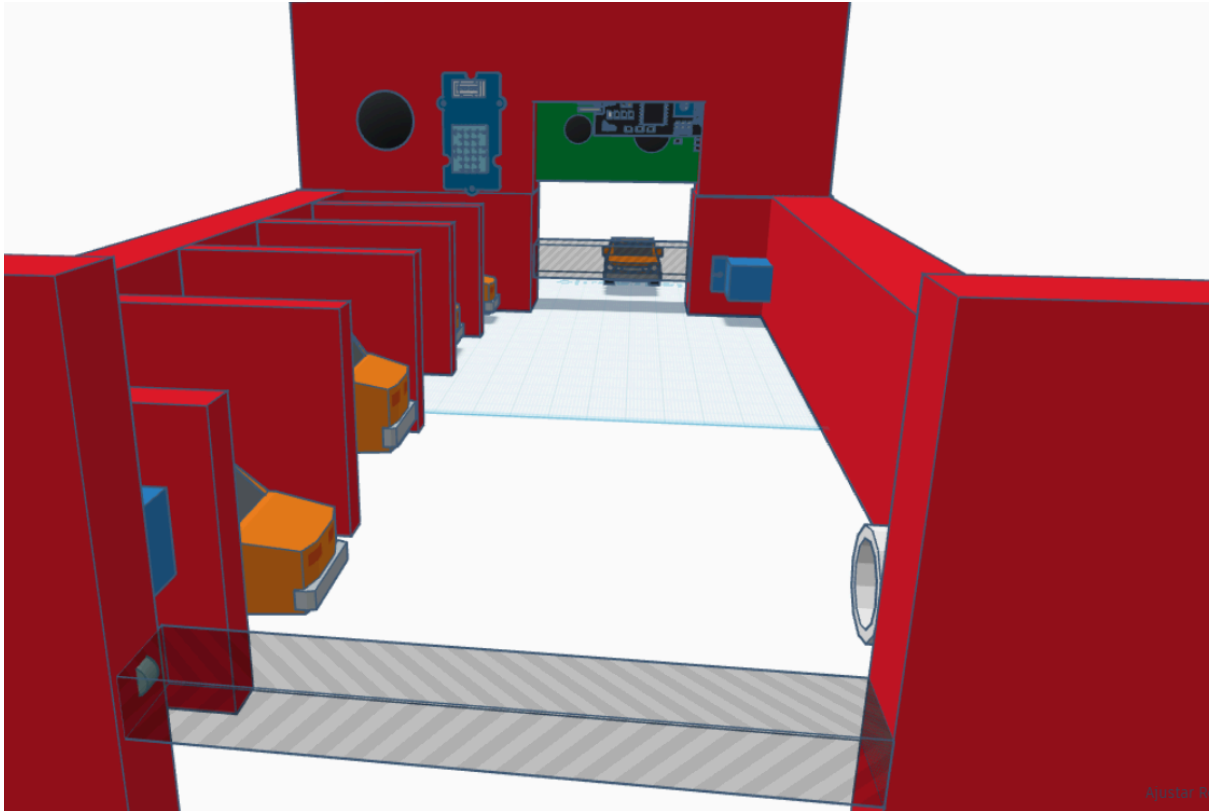
- SUPERIOR



- INTERIOR VISTA DE DELANTE



- INTERIOR VISTA DE ATRÁS



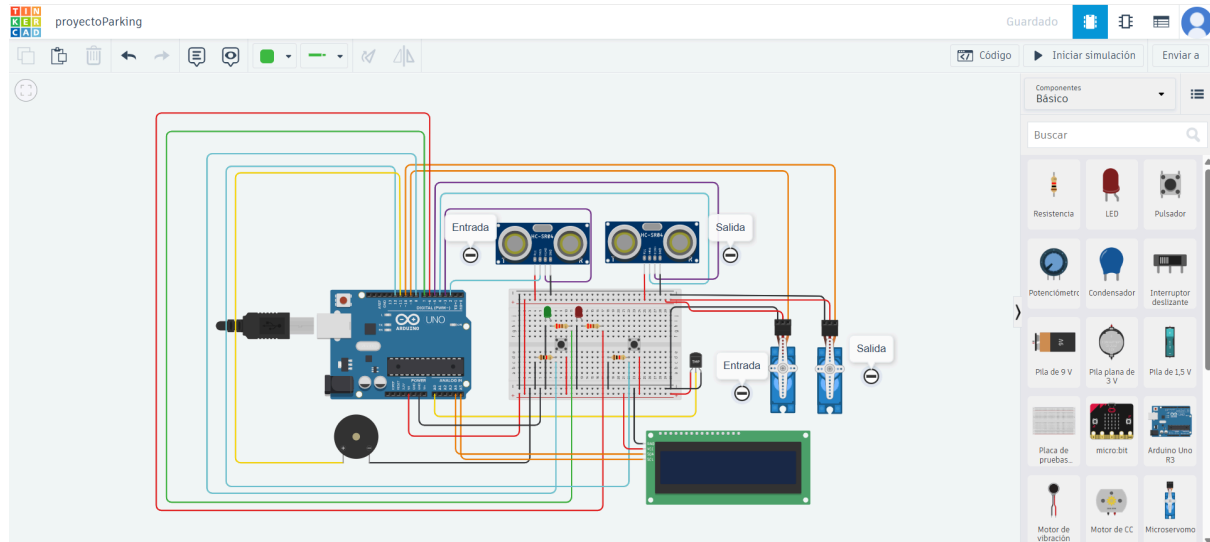
Podemos ver que el diseño se asemeja bastante a un parking real, donde se muestran los diferentes componentes de Arduino necesarios para su correcto funcionamiento. Para verlo con más detalle, usar el siguiente enlace:

https://www.tinkercad.com/things/af4uXQIPGKm-parkinginteligente?sharecode=5ZY4LSz_OCG4BdWrJCnJYBtXQg4LTyeygd7nNBrlbM

Parking montado con Tinkercad

Pasamos a la construcción del parking con el simulador Tinkercad, en donde se buscarán los componentes necesarios para montar el proyecto y se probará cuál es la mejor configuración de estos a la hora de desarrollar este parking.

Una vez seleccionados todos los componentes necesarios explicados en la introducción, nos quedaría de la siguiente forma el esquema.



La configuración de estos componentes se ha hecho a través de programación en texto, lo cual se ha dividido en dos métodos:

- **variables globales:** vamos a definir los valores que van a tener los diferentes pines colocados en la placa Arduino. Para ello, se ha hecho de la siguiente manera:
 - **Configuraciones de pines y dispositivos:**
 - **Servos:** debemos de tener 2 objetos Servo, uno que controle la entrada y otro la salida.
 - **Ultrasonido:** los usaremos para medir distancias en los puntos de entrada y salida. Estos se encontrarán en los pines 2,3 (para el ultrasonido de la entrada) y 4,5 (para el ultrasonido de la salida).
 - **LEDs:** indicarán visualmente el estado de la barrera de entrada (si está cerrado o abierto). Los configuramos en los pines 6 y 7 como salida, permitiéndonos poder enviar señales para encender y apagar los LEDs.
 - **Pulsadores:** tendremos 2 y serán los que se presionaran para poder abrir la barrera de entrada o salida. Configurados en los pines 12 y 8 como entrada para que pueda leer señales, como los datos que le transmita nuestro pulsador.
 - **Piezo:** encargado de sonar en caso de que la temperatura sea muy alta. Configurado en el pin 11.

- **Sensor de temperatura:** definimos el pin analógico A0 para leer el este sensor.
- **Mediciones de ultrasonido:** se definirán 2 variables de tiempo y 2 variables de distancia para calcular las distancias medidas por los sensores ultrasónicos.
- **Control de plazas:** usaremos una variable que actuará como contador de número de coches que hay en nuestro parking y otra variable que indique el número de total de plazas que hay disponible en este.
- **Lectura del sensor de temperatura:** tendremos 2 variables, una que nos guarda el valor bruto leído del sensor y otra que nos guarda el valor de la temperatura en grados Celsius.
- **Configuración y uso del LCD:** crearemos un objeto para el display de 16 columnas x 2 filas, el cual usaremos para mostrar información.
- **Control de eventos:**
 - Debemos de tener varias variables que nos garanticen que las acciones de conteos y actualizaciones de pantalla no se repitan por fallo de rebote de los pulsadores o por la detección de sensores.
 - Una variable que se encargará de controlar el estado de la entrada.
 - Una variable que nos evite múltiples activaciones en el proceso de salida.

El código con las variables configuradas queda de la siguiente manera.

```
Servo servoMotor;
Servo servoMotor2;

const int pinTrigger = 2;
const int pinEcho = 3;
const int pinTrigger2 = 4;
const int pinEcho2 = 5;
const int ledRojo = 6;
const int ledVerde = 7;
const int pulsador = 12;
const int pinPiezo = 11;
const int pulsadorE = 8;

const int pinTemp = A0;

float tiempo = 0;
float distancia = 0;
```

```
float tiempo2 = 0;
float distancia2 = 0;

int conta = 0;
const int totalPlazas = 5;

int sensorValor = 0;
float temperatura = 0;

Adafruit_LiquidCrystal lcd_1(0);

unsigned long tiempoUltimaEntrada = 0;
unsigned long tiempoUltimaSalida = 0;
const unsigned long retardoAntirrebote = 500;
unsigned long ultimaActualizacionPantalla = 0;
const unsigned long intervaloPantalla = 100;

int estadoEntrada = 0;

bool salidaActivada = false;
```

- **setup:** en este método nos encargaremos de realizar la configuración inicial de los componentes, el cual solo se ejecutará una sola vez al inicio del programa. Se ha realizado de la siguiente manera:
 - **Configuración de pines para sensores ultrasónicos:** definimos los sensores ultrasónicos con el método `pinMode()`, estableciendo que pines actuarán como salidas (para enviar señales) y cuáles como entradas (para recibir la información).
 - **Configuración de pines para servos:** meteremos los servos en los pines digitales 9 (barrera de entrada) y 10 (barrera de salida), iniciando ambas en posición cerrada.
 - **Configuración del LCD:** iniciamos la pantalla y posicionamos el cursor en ubicaciones concretas para mostrar los valores "P: " para las plazas disponibles y "T: " para la temperatura. También deberemos de limpiar la segunda línea del display para las modificaciones que hagamos en las diversas condiciones.
 - **Configuración de los pines de los LEDs:** configuramos ambos como salidas, dejando el LED rojo encendido y el LED verde apagado.
 - **Configuración de pulsadores:** usaremos resistencias internas para evitar lecturas erróneas.

- **Configuración del piezo:** lo definimos como salida para poder activarlo cuando la temperatura supere el umbral.

El método de configuración inicial queda como sigue.

```
void setup() {
  pinMode(pinTrigger, OUTPUT);
  pinMode(pinEcho, INPUT);
  pinMode(pinTrigger2, OUTPUT);
  pinMode(pinEcho2, INPUT);

  servoMotor.attach(9);
  servoMotor.write(0);

  servoMotor2.attach(10);
  servoMotor2.write(0);

  lcd_1.begin(16, 2);
  lcd_1.setCursor(0, 0);
  lcd_1.print("P: ");
  lcd_1.setCursor(8, 0);
  lcd_1.print("T: ");
  lcd_1.setCursor(0, 1);
  lcd_1.print("                ");

  pinMode(ledRojo, OUTPUT);
  pinMode(ledVerde, OUTPUT);
  digitalWrite(ledRojo, HIGH);
  digitalWrite(ledVerde, LOW);

  pinMode(pulsador, INPUT_PULLUP);
  pinMode(pulsadorE, INPUT_PULLUP);
  pinMode(11, OUTPUT);

  Serial.begin(9600);
}
```

- **loop:** este método será el que se encargue de ejecutar en bucle la lógica de control y monitoreo del parking. Se ha seguido una lógica la cual se ha dividido en varios pasos:

- **Captura del tiempo actual:** guardaremos el tiempo en milisegundos para poder usarlo en cálculos de tiempo, control de antirrebote y actualización de nuestra pantalla.
- **Lectura y conversión de la temperatura:** se lee el sensor de temperatura y almacenando el valor dado, convirtiéndolo a grados Celsius mediante la fórmula de temperatura que hay que usar para el sensor de temperatura tmp36.
- **Lectura del sensor ultrasónico de la entrada:** se envía que el pulsador ha sido presionado (establecido primero en LOW, luego en HIGH y luego en LOW mediante micro retrasos), midiéndose el tiempo que tarda el pulso en regresar. Además, calculamos la distancia con el factor de conversión que nos ayuda a detectar el vehículo a cuántos centímetros está.
- **Lectura del sensor ultrasónico de la salida:** realizamos un proceso similar al anterior, pero esta vez para el de salida.
- **Control de la puerta de entrada y actualización del contador de plazas:** se ha dividido en 2 partes, una si el sensor de temperatura indica que hay riesgo de incendio y el modo normal.
 - **Modo emergencia:** si la temperatura es mayor a 30°C (umbral de temperatura), se mostrará por la pantalla del LCD un mensaje de “INCENDIO”, manteniendo la puerta cerrada para que no puedan entrar más coches. Además, el piezo empezará a sonar
 - **Modo normal:** se verifica si hay plazas disponibles en el parking y, llevaremos a cabo 3 fases.
 - Si se detecta un vehículo cerca (menor o igual a 30 cm), pasamos a la siguiente fase de entrada.
 - En el estado de espera mostraremos por pantalla del LCD que el vehículo “PULSE BOTON”. Si se pulsa el botón de entrada, se comprobará el tiempo transcurrido para evitar rebotes, pasando a la última fase.
 - Mostraremos el mensaje de bienvenida al vehículo y abriremos la barrera de entrada (girar el servo 90 grados), encendiendo el LED verde y apagando el rojo. Incrementaremos el contador de vehículos y finalmente, cerraremos la barrera de entrada, restableciendo los LEDs y el estado de entrada.
- **Control de la puerta de salida y liberación de plazas:** al igual que en la entrada, esta parte se dividirá en 2, en el caso de que el sensor de temperatura indique que hay riesgo de incendio y el caso normal.
 - **Modo emergencia:** cuando se indique una temperatura mayor a 30°C, se mostrará por pantalla del LCD un mensaje de “INCENDIO”, abriendo la barrera de salida para que los coches puedan salir. Se irá decrementando el contador de plazas conforme los vehículos vayan saliendo del parking, sin necesidad de presionar el pulsador.

- **Modo normal:** la barrera de salida se abrirá (servo se gira 90 grados) si el vehículo presiona el pulsador de salida y se detecta cerca de la barrera. Finalmente, una vez el coche salga del parking, la barrera de salida se cerrará y se indicará que hay una plaza más disponible en nuestro parking.
- **Actualización del LCD:** la primera línea muestra todo el rato la pantalla LCD comentará el número de plazas disponibles que hay actualmente en el parking, actualizándose conforme entre o salga un vehículo de este. La segunda línea muestra mensajes según la situación.
 - **INCENDIO:** si la temperatura mostrada por el sensor es superior a 30 grados.
 - **PULSE BOTON:** si un coche se encuentra cerca de la barrera de entrada.
 - **BIENVENIDO:** cuando el vehículo presione el pulsador y se detecte cerca se mostrará el mensaje de bienvenida.
 - **Parking Completo:** se indica cuando ya no hay más plazas disponibles en el parking.

El código de la lógica principal del proyecto del parking es el siguiente.

```
void loop() {
  unsigned long millis = millis();

  sensorValor = analogRead(pinTemp);
  temperatura = (sensorValor * (5.0 / 1023.0) - 0.5) * 100.0;

  digitalWrite(pinTrigger, LOW);
  delayMicroseconds(4);
  digitalWrite(pinTrigger, HIGH);
  delayMicroseconds(10);
  digitalWrite(pinTrigger, LOW);
  tiempo = pulseIn(pinEcho, HIGH);
  distancia = 0.0172 * tiempo;

  digitalWrite(pinTrigger2, LOW);
  delayMicroseconds(4);
  digitalWrite(pinTrigger2, HIGH);
  delayMicroseconds(10);
  digitalWrite(pinTrigger2, LOW);
  tiempo2 = pulseIn(pinEcho2, HIGH);
  distancia2 = 0.0172 * tiempo2;

  if (temperatura > 30) {
    lcd_1.setCursor(0, 1);
    lcd_1.print("INCENDIO      ");
  }
}
```

```

servoMotor.write(0);
}
else if (conta < totalPlazas) {
    if (distancia <= 30) {
        if (estadoEntrada == 0) {
            estadoEntrada = 1;
        }
        if (estadoEntrada == 1) {
            lcd_1.setCursor(0, 1);
            lcd_1.print("PULSE BOTON    ");
            if (digitalRead(pulsadorE) == HIGH && (millis() -
tiempoUltimaEntrada >= retardoAntirrebote)) {
                estadoEntrada = 2;
                tiempoUltimaEntrada = millis();
            }
            servoMotor.write(0);
        }
        if (estadoEntrada == 2) {
            lcd_1.setCursor(0, 1);
            lcd_1.print("BIENVENIDO    ");
            servoMotor.write(90);
            digitalWrite(ledRojo, LOW);
            digitalWrite(ledVerde, HIGH);
            delay(2000);
            conta++;
            servoMotor.write(0);
            digitalWrite(ledRojo, HIGH);
            digitalWrite(ledVerde, LOW);
            estadoEntrada = 0;
        }
    }
}
else {
    estadoEntrada = 0;
    servoMotor.write(0);
}
}
else {
    servoMotor.write(0);
}

if (temperatura > 30) {
    digitalWrite(11, HIGH);
    servoMotor2.write(90);
    if (distancia2 <= 30 && !salidaActivada && (milis -
tiempoUltimaSalida >= retardoAntirrebote)) {
        if (conta > 0) {

```

```

        conta--;
    }
    salidaActivada = true;
    tiempoUltimaSalida = millis;
}
else if (distancia2 > 30) {
    salidaActivada = false;
}
}
else {
    if (distancia2 <= 30 && digitalRead(pulsador) == HIGH) {
        servoMotor2.write(90);
        delay(1000);
        if (!salidaActivada && (millis - tiempoUltimaSalida >=
retardoAntirrebote)) {
            if (conta > 0) {
                conta--;
            }
            salidaActivada = true;
            tiempoUltimaSalida = millis;
        }
    }
    else {
        digitalWrite(11, LOW);
        servoMotor2.write(0);
        salidaActivada = false;
    }
}
}
}

```

Para ver el proyecto desde Tinkercad, usar el enlace:

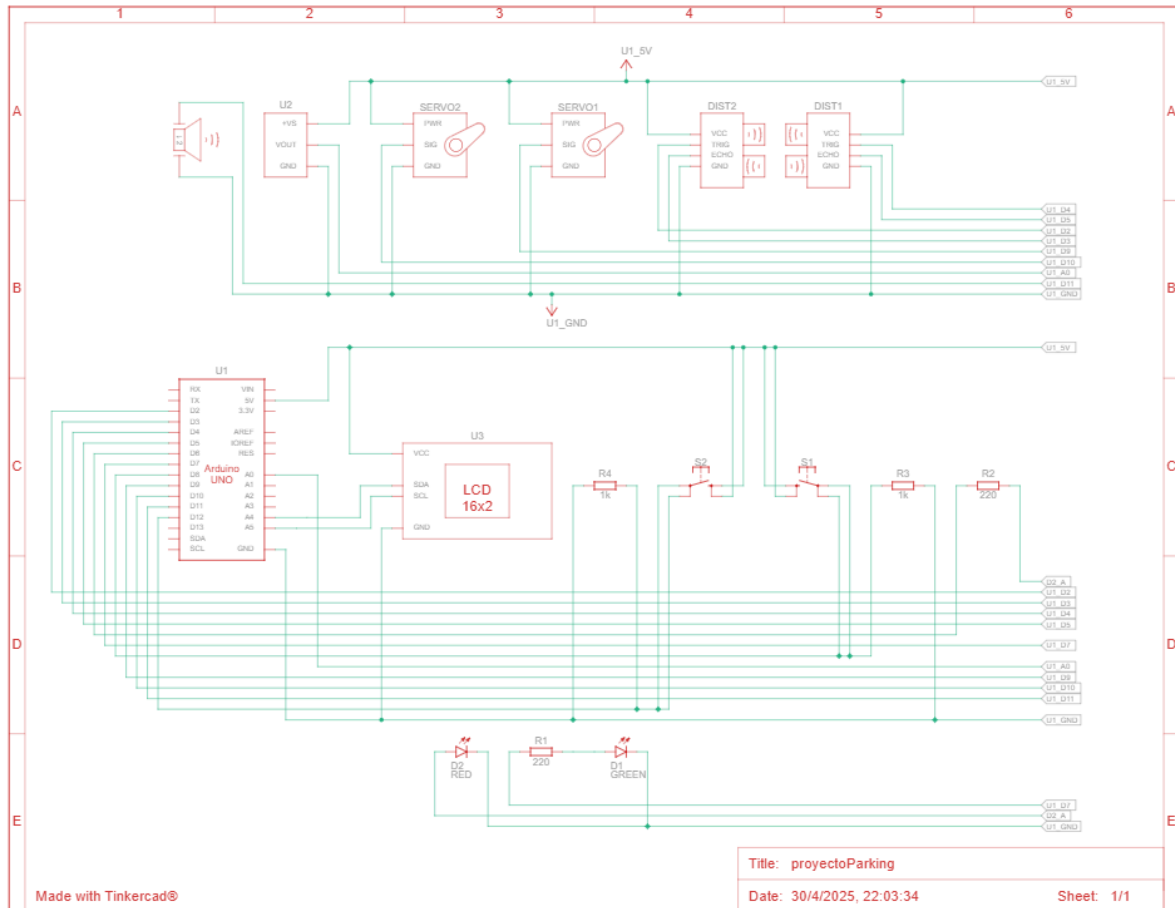
<https://www.tinkercad.com/things/c0NpnarMbod-proyectoparking?sharecode=gaeExCN81uwS8Gzd4wZdz9geQwAbOKtABSiGzSuS1dg>

Para ver el video representativo, usar el enlace:

https://drive.google.com/file/d/1C25aocSX86SofBAqWM2FUOW7UbsSfe2e/view?usp=drive_link

Esquema Fritzing

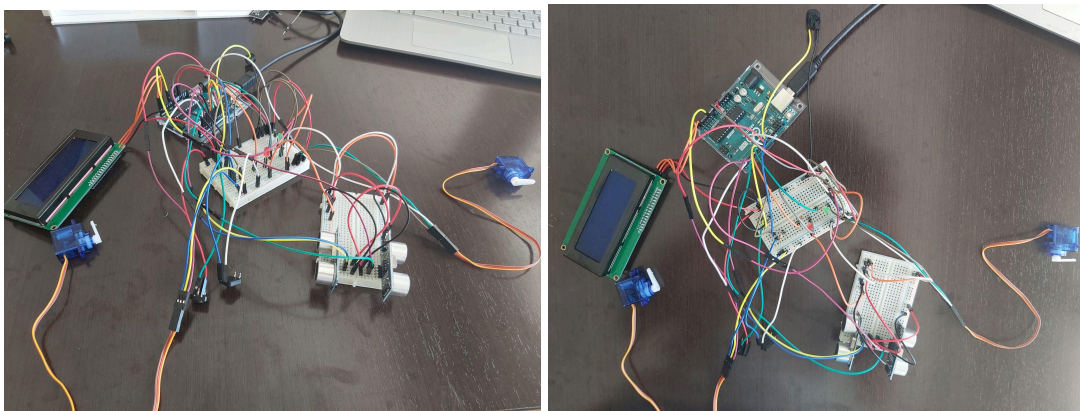
El esquema Fritzing del parking montado es el siguiente.



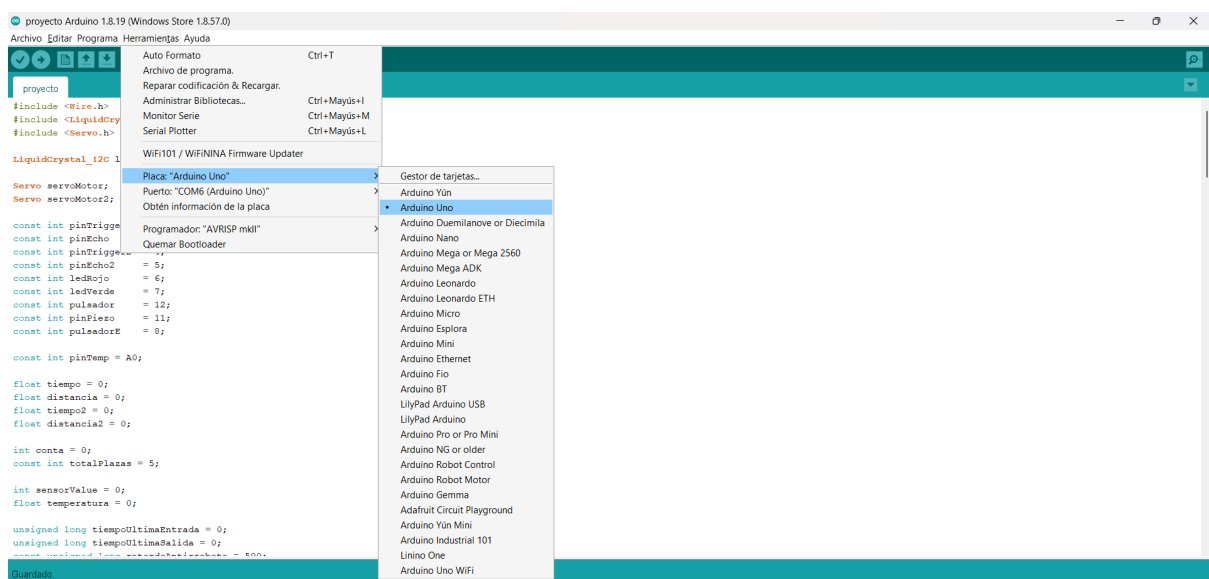
Parking montado físicamente con Arduino IDE

A la hora de montar el parking físicamente vamos a usar el programa Arduino IDE para poder implementar el código del proyecto. Se ha tenido que modificar ligeramente el código dado a que en la pantalla LCD no se visualizaban correctamente los mensajes, por lo que hemos usado una librería externa para ello. Por otro lado, se ha modificado el manejo del piezo para que genere un tono de 4000 Hz y se oiga bastante fuerte. No obstante, la estructura y lógica sigue siendo la misma.

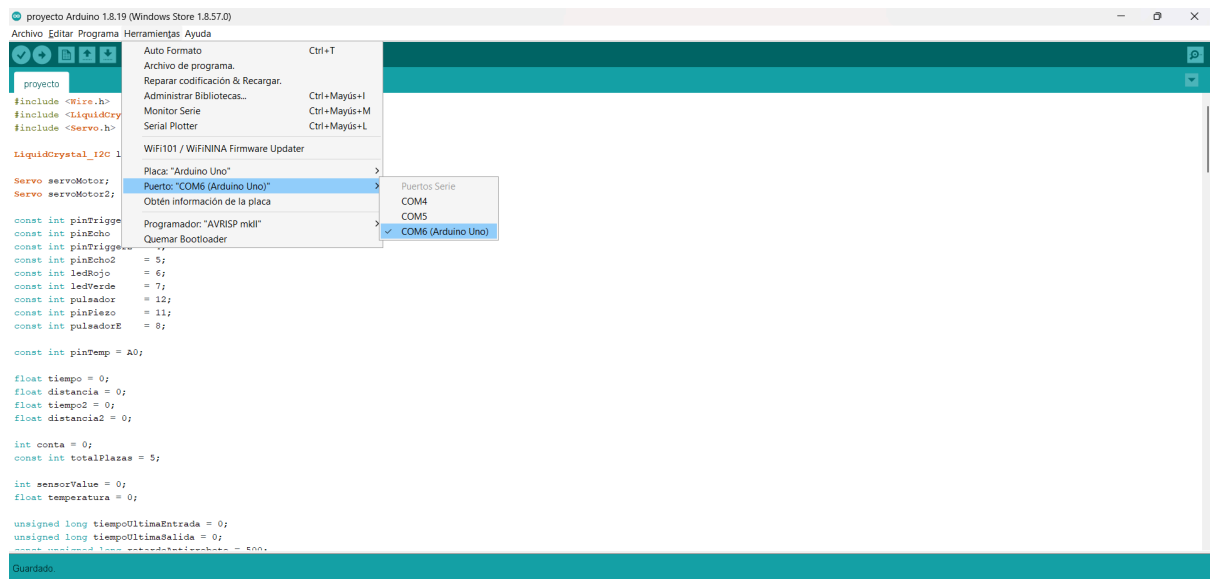
Lo primero de todo es realizar el ejercicio, esta vez, con dos placas de pruebas y con un *Arduino UNO R3*, montándolo con los componentes correspondientes. El ejercicio montado quedaría de la siguiente manera:



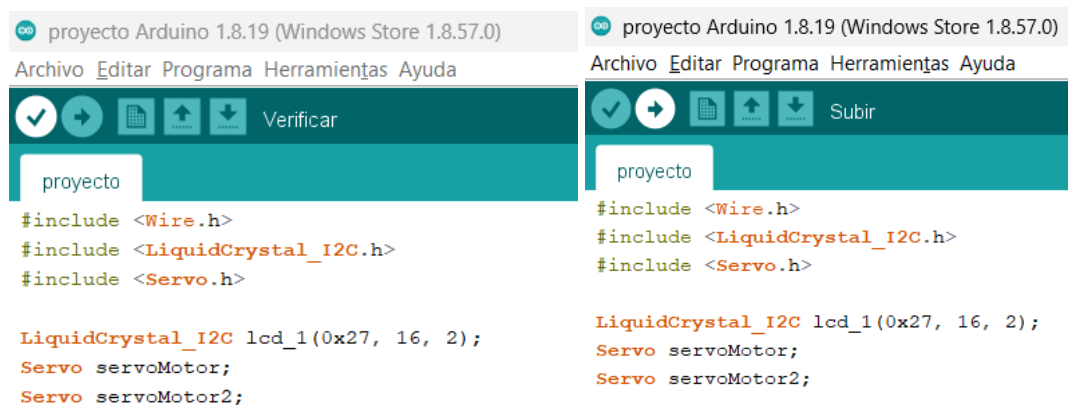
Ahora, pasamos a configurar dicho programa para que se conecte a la placa. Nos vamos a la parte del menú donde pone Herramientas y en *Placa* seleccionamos *Arduino Uno*.



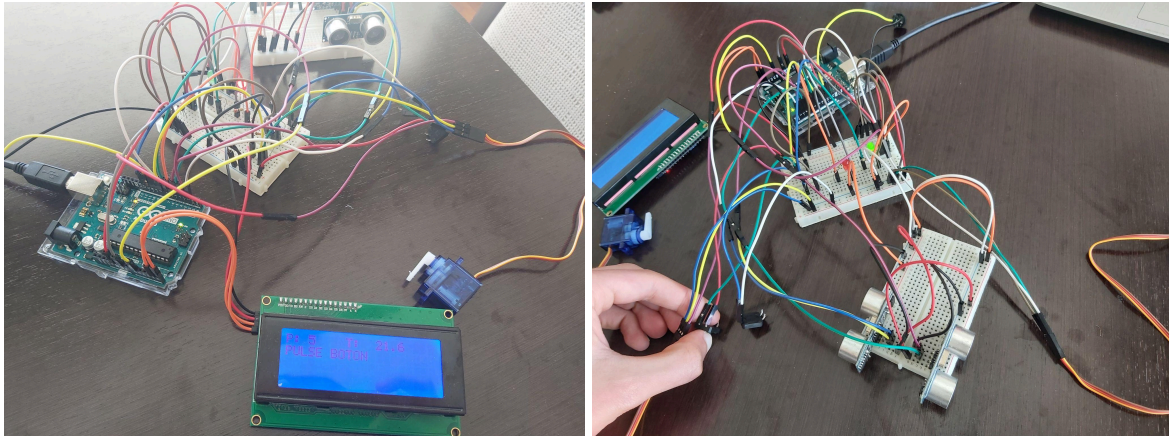
Además, cuando conectemos la placa, debemos de especificar el puerto que se va a utilizar (COM6 Arduino 1).



Luego, debemos de cargar el programa. Para ello primero debemos de pinchar el el botón de *Verificar* que está en la parte superior izquierda y luego a *Subir* (al lado de *Verificar*) para que el programa empiece a funcionar en la placa.



Una vez verificado y subido a la placa, veremos como el proyecto empieza a funcionar.

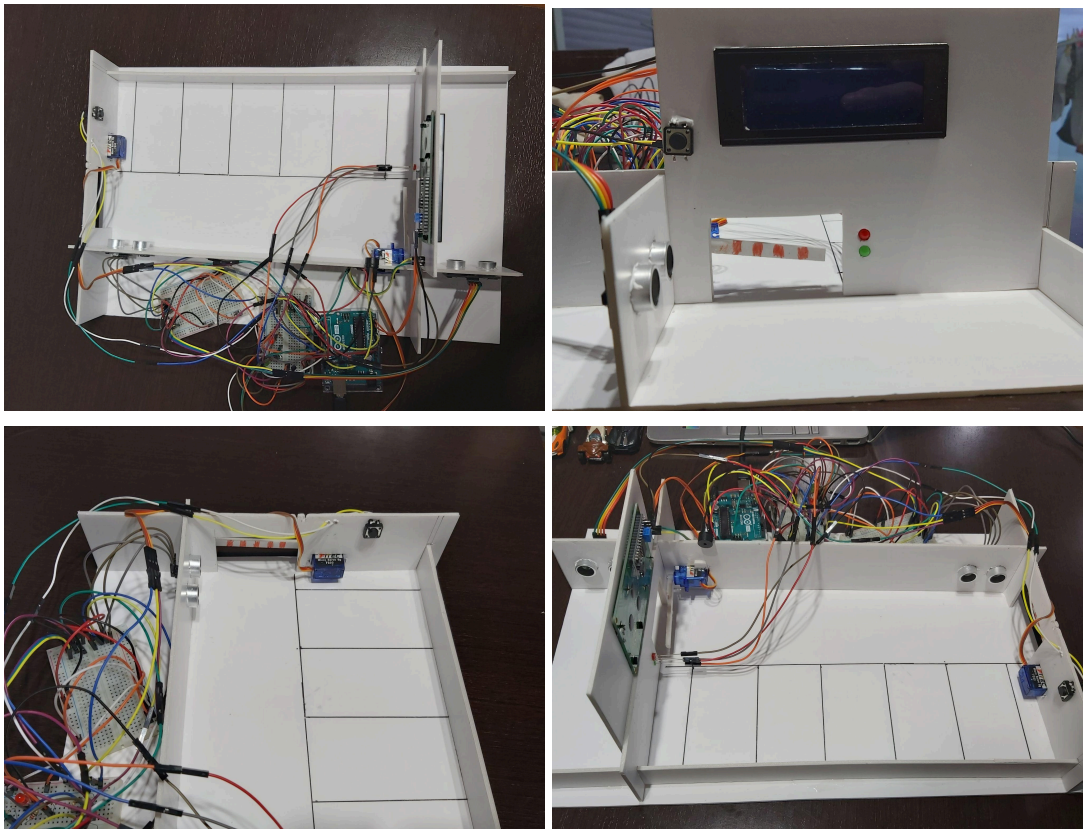


Para ver el video funcionando físicamente el proyecto, usar el enlace:

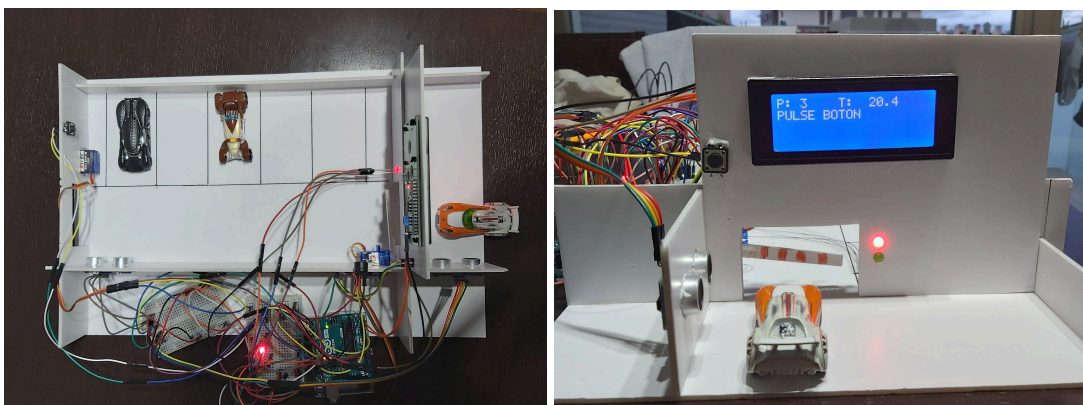
- Funcionamiento estándar:
https://drive.google.com/file/d/1yKb_PNImpfqBPpw9rO6K9iRETrFal9C/view?usp=drive_link
- Funcionamiento si hubiera incendio:
https://drive.google.com/file/d/1yQFkWEGWAwgYVsJ7jIArGGalQh-UI4ga/view?usp=drive_link

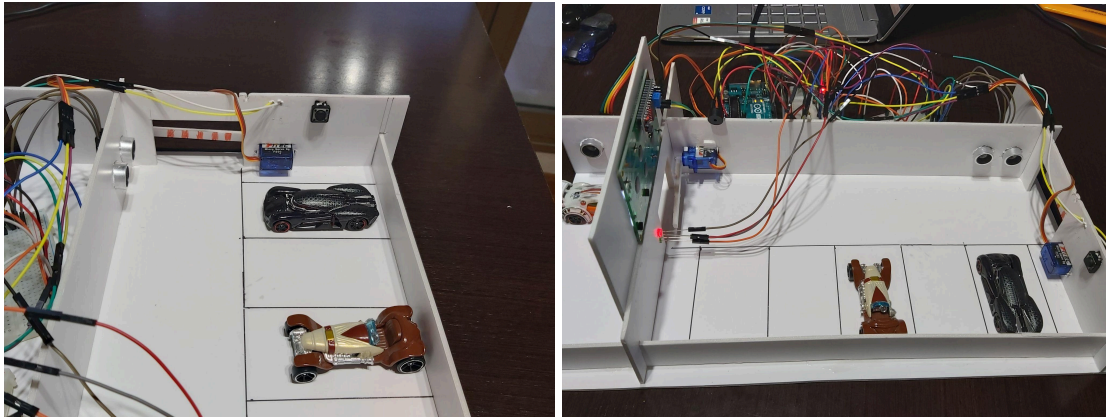
Maqueta y funcionamiento

Para ver el funcionamiento del proyecto en un entorno real simplificado, se ha realizado una maqueta de un parking similar a la del diseño en 3D creado en Tinkercad. Para ello se ha necesitado papel pluma para construir las paredes, además de hacer cortes precisos para introducir los diversos componentes de Arduino en la maqueta. Las dos placas de pruebas y la placa Arduino UNO R3 se han metido en un espacio tapado, proporcionando mediante cables la corriente necesaria para que funcione correctamente todos los componentes en la maqueta. Se ha usado el programa de Arduino IDE para cargar el código realizado. La maqueta ha quedado de la siguiente forma.



Una vez le damos a Subir en el programa Arduino IDE, empezará a funcionar.





Para ver el video funcionando físicamente el proyecto, usar el enlace:

- Funcionamiento estándar:
https://drive.google.com/file/d/1zqv4GpOFTMGAXvGYd3BDaOeKoXtjJFAX/view?usp=drive_link
- Funcionamiento si hubiera incendio:
https://drive.google.com/file/d/1zuThl5jw_LCRj628x1iyhgllNKXrYBd5/view?usp=drive_link

Conclusiones

A través de la realización de este proyecto de parking con Arduino y Tinkercad he descubierto a la hora de realizarlo el la utilidades que presenta la integración de la electrónica y la programación. Gracias a este proyecto he podido ver bastantes componentes (sensores de distancia, sensores de temperatura, servomotores, etc) para entender la importancia de su correcta configuración y sincronización para lograr un sistema automatizado fluido, dado que uno de los principales objetivos de un parking es eso, sincronización y actualización constante.

Al principio ha sido un reto controlar con precisión las barreras de entrada y salida en tiempo real, haciéndome ver la complejidad de lograr que cada elemento funcione correctamente en cada momento. Cada sensor y actuador han sido de gran relevancia en todo momento: los sensores ultrasónicos detectaron si había vehículos cerca, los pulsadores iniciaron secuencias de operación, los LEDs se sincronizaban cuando se abría la puerta de entrada, los servomotores mostraron la apertura y cierre de barreras en respuesta a estos estímulos, así como el sensor de temperatura y el piezo, en donde si la temperatura ambiente supera el umbral, el buzzer empezaría a sonar. Esta coordinación nos muestra la calidad de que cualquier solución automatizada depende de la capacidad para sincronizar múltiples variables en tiempo real.

Además, el uso de Arduino me ayudó para experimentar con estos dispositivos y su integración, además de mostrarme una base sólida para abordar proyectos de mayor complejidad y aplicarlos en problemas reales. La simulación en Tinkercad me ayudó para validar y perfeccionar la lógica de control de forma virtual, minimizando errores y optimizando el rendimiento del sistema antes de llevarlo a una implementación práctica con Arduino IDE, el cual gracias a la experimentación con el simulador no hubo ningún problema. Además, la creación de una maqueta para ver de una forma más visual el proyecto nos hace pensar en la importancia de plasmar los proyectos reales en pequeños esbozos para ver cuál sería el funcionamiento para luego aplicarlo a problemas reales.

Este proyecto, aparte de ser un ejercicio práctico, también me ha servido para entender la importancia de la automatización, la correcta sincronización. He podido observar cómo partiendo de conceptos básicos, es posible construir soluciones innovadoras, adaptables y óptimas para resolver problemas actuales (como la gestión eficiente de un parking) y de gran importancia en nuestra actualidad.

Bibliografía

<https://www.youtube.com/watch?v=2918sSKOghc>

<https://www.youtube.com/watch?v=HQqwLuJ1pbo>